

MultiThreaded Proxy Web Server Project

1. Introduction to the Project

This project implements a fully functional **multi-threaded HTTP proxy server with caching**. A proxy server acts as an intermediary between a client (such as a browser) and a remote web server. Instead of contacting the web server directly, the client sends its request to the proxy. The proxy then forwards this request, receives the response, and finally delivers the response back to the client. Your proxy also maintains an internal cache so frequently requested pages can be served immediately, improving speed and reducing network traffic. In summary, this system demonstrates how real-world proxies manage connections, parse HTTP requests, interact with remote servers, and optimize performance through caching.

2. Understanding HTTP Request Parsing

A critical component of a proxy is its ability to understand and validate incoming HTTP requests. This is handled by the `proxy_parse` module. When a client sends an HTTP request such as:

```
GET http://example.com/page HTTP/1.1
Host: example.com
Connection: keep-alive
```

the parser breaks this raw string into meaningful components. It extracts the **method** (GET), **protocol** (http), **host**, **port**, **path**, and **HTTP version**. It also records all other HTTP headers in a structured format. By organizing the request into clearly defined fields, the proxy can validate the request, modify headers, and rebuild a clean and correct request to forward to the target server. The parser also ensures that the request complies with HTTP formatting rules, and rejects malformed requests to maintain server safety and correctness.

3. Socket Programming and Network Communication

Your proxy server uses low-level socket programming to communicate over the network. It begins by creating a TCP socket, binding it to a port (often 8080), and listening for incoming

client connections. When a client connects, the proxy accepts the connection and receives the HTTP request using system-level functions like `recv()` and `send()`. Once a request is parsed, the proxy opens another socket and initiates a connection to the remote server specified in the request. It forwards the cleaned-up HTTP request to this server and receives the server's response in chunks. These chunks are immediately sent back to the client. The entire process mimics how browsers communicate with web servers, except your proxy sits in the middle, processing and forwarding data in both directions.

4. Multithreading and Concurrency Management

The proxy server supports multiple simultaneous clients using **POSIX threads (pthreads)**. When a new client connection is accepted, the proxy creates a new thread to handle that client independently. This ensures that multiple clients can be serviced in parallel without blocking each other. To ensure the server does not become overloaded, a **semaphore** limits how many threads may run concurrently. If too many clients connect, new threads wait automatically until existing requests finish. This approach prevents resource exhaustion and ensures the proxy remains responsive even under heavy load. In essence, the project demonstrates how to build scalable, concurrent network services using multi-threading.

5. Cache Design and LRU Eviction Policy

One of the most important features of the project is the implementation of a **cache** using a linked list of `cache_element` structures. Each cached element stores the full URL (used as a key) and the entire HTTP response (used as the cached value). To keep the cache efficient and within memory limits, it follows the **Least Recently Used (LRU)** eviction policy. Every time a cached element is accessed, its timestamp is updated to the current time. When new data needs to be cached and the cache is full, the proxy scans the cache to find the oldest (least recently accessed) entry and removes it to free space. This ensures the cache remains populated with useful, frequently requested content while discarding outdated or rarely accessed responses.

6. Thread Safety Through Mutex Locks

Because many threads may read from or write to the cache concurrently, the project incorporates strict thread synchronization using **mutex locks**. Only one thread is allowed to modify or read from critical cache structures at a time. This prevents race conditions, data corruption, and invalid memory access. Locking ensures that the cache remains consistent and

reliable no matter how many clients are interacting with the proxy. This demonstrates proper use of concurrency control mechanisms—a vital aspect of systems programming.

7. Full Request–Response Processing Flow

The lifecycle of a client request in your proxy can be summarized as follows:

1. A client connects and sends an HTTP request to the proxy.
2. The proxy receives the raw request and checks whether this request already exists in the cache.
3. If the response is found in the cache, it is immediately returned to the client.
4. If not found, the proxy parses the HTTP request and validates its structure.
5. The proxy reconstructs a clean GET request, ensuring required headers like `Connection: close` and `Host` are present.
6. A connection is made to the remote server, and the request is forwarded.
7. The proxy receives the response in chunks and streams it back to the client.
8. Simultaneously, the response is copied and stored in the proxy's cache for future reuse.
9. The connection to the server and client is closed, and the thread terminates.

This flow represents a complete end-to-end handling of HTTP communication through a proxy.

8. Error Handling

The proxy includes explicit handling of several HTTP error conditions. If a malformed request is detected, or a request includes an unsupported HTTP method or version, the proxy returns appropriate error pages such as 400 (Bad Request), 403 (Forbidden), or 500 (Internal Server Error). These responses ensure the client receives meaningful feedback and prevent the proxy from forwarding invalid traffic to remote servers.

9. Summary of What This Project Teaches

This project offers deep insight into the internal workings of modern networking systems. You learn how HTTP requests are structured and parsed, how TCP sockets handle bidirectional communication, and how concurrent servers scale through multi-threading. You also gain significant experience with caching algorithms, memory management, synchronization primitives, and request reconstruction. In combining these elements, the project mirrors the architecture of real proxy servers used for load balancing, caching, privacy, and traffic routing across the internet.





